

Practical Week 12

Task 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

Step 1: Create the Producer-Consumer program

nano producer_consumer.c

```
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define ITEMS 1000

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}
```

```

GNU nano 7.2 producer_consumer.c *
void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("Producer-Consumer completed successfully.\n");

    return 0;
}

```

Step 2: Compile and Run

```

bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ cd ~/OSSP/Week12
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ nano producer_consumer.c

bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc producer_consumer.c -o producer_consumer -pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ ./producer_consumer
Producer-Consumer completed successfully.
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ |

```

Step 3: Verify synchronization

```
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ ./producer_consumer
./producer_consumer
./producer_consumer
Producer-Consumer completed successfully.
Producer-Consumer completed successfully.
Producer-Consumer completed successfully.
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ |
```

Step 4: Test Different Buffer Sizes

```
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc producer_consumer.c -o producer_consumer -
pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ /usr/bin/time -f "Time: %e seconds" ./producer
_consumer
Producer-Consumer completed successfully.
Time: 0.02 seconds
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ nano producer_consumer.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc producer_consumer.c -o producer_consumer -
pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ /usr/bin/time -f "Time: %e seconds" ./producer
_consumer
Producer-Consumer completed successfully.
Time: 0.02 seconds
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ nano producer_consumer.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc producer_consumer.c -o producer_consumer -
pthread
/usr/bin/time -f "Time: %e seconds" ./producer_consumer
Producer-Consumer completed successfully.
Time: 0.00 seconds
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$
```

Observation:

The Producer-Consumer problem was implemented using POSIX threads, counting semaphores, and a mutex. Buffer sizes of 5, 10, and 20 were tested. All tests completed successfully, with execution times of 0.02 s, 0.02 s, and 0.00 s respectively.

Result:

Synchronization was verified successfully, preventing buffer overflow, underflow, and race conditions. Changing the buffer size had only a small effect on the measured execution time for this test.

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering

Step 1: Create the deadlock program

nano deadlock.c

```
GNU nano 7.2                                deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}
```

```

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Step 2: Compile and Demonstrate Deadlock

```

bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ nano deadlock.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc deadlock.c -o deadlock -pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ ./deadlock
Thread 2: locking Resource 2
Thread 1: locking Resource 1
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1
^C
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ |

```

Step 3: Prevent Deadlock Using Resource Ordering

nano deadlock_prevention.c

```
GNU nano 7.2                                deadlock_prevention.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}
```

```

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevention successful.\n");

    return 0;
}

```

Step 4: Compile and Test the Deadlock Prevention

```

bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ nano deadlock_prevention.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$ ./deadlock_prevention
Thread 1: locking Resource 2
Thread 1: acquired both resources
Thread 2: locking Resource 1
Thread 2: locking Resource 2
Thread 2: acquired both resources
Deadlock prevention successful.
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week12$

```

Observation:

A deadlock scenario was created using two threads and two shared resources. The threads held one resource while waiting for the other, demonstrating mutual exclusion, hold and wait, no preemption, and circular wait. Resource ordering was then applied so that both threads acquired resources in the same order.

Result:

Deadlock was successfully demonstrated and prevented using resource ordering. By eliminating the circular-wait condition, both threads completed successfully without becoming permanently blocked.

